Assignment 2 - Cheap Shadows, Shadow Maps, and Post Perspective Viewer

In this assignment you will implement shadows for object order rendering. This includes shadow maps and cheap shadow projections.

Getting Started

You can use the same python and environment that you set up in the previous assignment as this assignment has the same module requirements.

Provided Code

The provided code sets up four views of the same scene: the main view, the light view, the second camera view, and the post-perspective view of the main camera. Each has a camera that can be controlled via mouse motion (XYBall) and mouse wheel (move closer farther from world origin). The cameras are defined in `Scene.py`, which also holds and preps the geometry of the scene objects for drawing. See that the `ViewSceneControlWidget.py` wrangles the painting of all four views and also calls the scene to render the shadow map to a texture for it to be used in drawing the different views. Each view's paint method will set up the camera view and projection matrices, but will not quite work as desired until you implement the different objectives.

Notice that the scene also has controls, both in the form of a UI, but there are also keyboard controls.

While you should look through all the code to familiarize yourself with the different parts, you will note that there are TODO comments that help you quickly find the relevant locations where you will need to make changes for the different objectives. For example, while there are no TODO comments in `ViewLight.py`, you should notice how it makes calls for computing the frustum that you will need to implement in `Scene.py`, which contains corresponding TODO comments.

Finally, notice that all the scene geometry is defined in world coordinates, so there are no modeling transformations associated with drawing this geometry. Nevertheless, there are parts of the assignment where you will need to think about modeling transformations!

Objectives

This assignment has several steps where the marks associated with different steps are below. There are 13 points total.

1. **(1 marks) Compute Light Position.**
    - The light position is defined by the `light_view_camera` which is used by `ViewLight.py` (see that `ViewSceneControlWidget.py` provides the appropriate camera to each view). The light position is needed in both world coordinates, and in the coordinates of other viewing frames. Implement `Scene.get_light_pos_in_world`, and notice that lighting in your main view should make more sense with respect to the light position as set by mouse interactions in the light view.

2. **(2 marks) Draw Light Frustum in Second View.**

   - The second camera view (bottom left of the 4 views) exists so that you can view the scene along with both the light and main camera frustums and their corresponding view frames. Compute the appropriate modelling transformation matrix to draw the frustum using the call to draw [-1,1] wire cube. Similarly set up an appropriate matrix for drawing the coordinate frame of the light viewing frame in the second view.

3. **(2 marks) Draw Main Camera Frustum in Second View.**

   - Similar to the previous objective, compute the appropriate transformations for drawing the camera frustum and frame in the second view.

4. **(2 marks) Cheap Shadows.**

   - Compute a transformation matrix that depends on the light position and the ground plane, and projects geometry onto the ground plane. These cheap shadows should only be drawn in the main view, and you see a TODO flagging this objective in `ViewMain.py` .
   - Use the technique seen in class to construct the projection, and include a small offset to ensure your shadows are drawn slightly above the ground plane (i.e, 0.001 units above the plane).

5. **(2 marks) Shadow Maps.**

   - The paint method of `ViewSceneControlWidget.py` calls the `Scene.render_shadow_pass` to draw the shadow map for the light view, but this method needs the correct matrices to be set up. You need to set the `u_mvp` matrix, i.e., combined modeling, viewing, and projection transformation matrix, for drawing the light view's depth. You also need to set up `u_light_space_transform` , i.e., the light space transform that takes vertices in world coordinates to texture coordinates in the shadow map.
   - You may wish to look at the GLSL code to solidify your understanding from class with respect to how these transformations are used to compute quantities that ultimately permit the drawing of shadows.

6. **(2 marks) Tight frustums**.

   - The provided code has default near and far planes which can be improved, and a manual mode for adjusting the light field of view. You may note that increasing the field of view spreads out the shadow map over a larger area of the world, and generally leads to a poorer quality shadow. Thus, tight bounds are preferable!
   - Compute tight near and far planes for the provided view in the provided `scene.compute_nf_from_view` function. This is called during setup of the main camera and the light camera. See that the scene has a complete list of vertices that you can examine to determine these bounds (i.e., transform to the view frame and compute bounds).
   - Compute tight lrbt for the light camera by completing the `scene.compute_lrbt_for_projection` function. You can find these values by computing bounds on vertices once they have been transformed by both the viewing matrix, and by a projection matrix which does not include the windowing transformation seen in class (i.e., find a bound on the vertices post projection, but before mapping them with a windowing transformation to the canonical viewing volume).

7. **(2 marks) Main Camera Post Projection View**.

   - Drawing code for the post projection view is provided. It includes a reflection in Z after the camera projection to make lighting easier (i.e., this reflection avoids the flip from a right-handed to a left-handed coordinate system which is the standard form used in OpenGL and discussed in class).

- Add code to compute the appropriate matrices to draw the CCV coordinate frame (i.e., at the middle of the [-1,1] cube), along with the light camera frustum, light camera view frame, and main camera frustum. Can you explain why it is difficult to draw the main camera view frame? Leave a comment in the code with respect to this.

Finished?

Great! Submit your python implementation and log file, and optionally a readme file as a `zip` archive to MyCourses (only use `zip` format). Use the comments at the top of the files you submit to list your name and student number. Recall the readme can be used to request a late penalty waiver following the mechanism described in the first lecture slides.

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.